

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D055.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 055 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

调试技巧:ipdb

// Debugging

KEY CONCEPTS · 三大要点

- ▶ set_trace
- ▶ postmortem
- ▶ logging

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面几节我们写的代码越来越长,bug 也越来越多。这一节专门教你怎么抓 bug。很多人遇到程序出错,第一反应就是到处加 print,满屏幕打数,跟车坏了满大街喊『哪儿坏了』一样,又累又看不出名堂。这一节请出两件正经工具:第一是 ipdb,它能让程序在你指定那一行『停下来』,像把车架起来,一个零件一个零件地查,出错那一刻冻住现场让你一行行看;第二是 logging,它比随手 print 优雅多了,像行车记录仪,可以分等级、可以一键关掉。我们会用京东方A的真实数据,亲手抓一个回测里最常见也最致命的 bug:未来函数,让你看清它怎么把回测净值吹成假的几十倍,又怎么被调试工具一招打回原形。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 学会调试的正确姿势:不是到处 print,而是让程序在出错那行停下来,逐行检查现场
- 02 认识 ipdb 和它的 set_trace 下断点、postmortem 出错后现场勘查两个核心用法
- 03 学会用 logging 分级日志代替满屏 print,信息可分级、可一键关掉、更干净
- 04 亲手抓一个未来函数 bug:看清回测怎么偷看明天的价格,把净值吹成假的几十倍
- 05 学会用断言 assert 和 try/except + traceback 给程序兜底,离谱结果第一时间现形

02 · CONTEXT · 历史与背景

老史的回测年化几十倍乐疯了,上实盘亏到怀疑人生

// 不读历史的策略走不远

老史自学了点 Python,写了个回测,跑出来的结果把他自己都看傻了:几年时间净值翻了几十倍,年化高得离谱。他兴奋得一晚上没睡,觉得自己发现了印钞机,第2 天就把大半身家投了进去上实盘。结果呢?实盘一上来就连续亏,跟回测里那条冲天的曲线完全是两个世界,几个月就亏得他怀疑人生。老史百思不得其解,代码明明一字没改,怎么回测赚翻、实盘巨亏?后来他硬着头皮回去逐行调试,在关键那行停下来一看,傻眼了:他的买入信号里,不小心用到了第2 天的收盘价,也就是说,程序在做今天的决策时,偷偷看了明天的答案。这就是回测里最经典、也最致命的 bug,叫未来函数。现实里你根本拿不到明天的价格,这种偷看未来算出来的几十倍,全是镜花水月。老史的教训是:回测结果好到不真实,先别高兴,八成是 bug,而抓出它,靠的不是到处 print,是停下来逐行看现场。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

断点调试:让程序停在你怀疑的那一行

断点调试,就是在代码里你怀疑有问题的那一行做个记号,叫『断点』,程序跑到这里就会乖乖停下来,把那一刻所有变量的值都冻在那里给你看。打个比方:车开着开着觉得不对劲,你不会让它一直开,而是开到路边停下、架起来慢慢查。比如你怀疑某个变量 `signal` 算错了,就在算它的下一行设个断点,程序停下后你可以现场敲 `signal`,看看它到底等于几。这比到处加 `print` 强太多,因为你能在停下的那一刻,想看什么变量就看什么变量。

CONCEPT_02

ipdb 与 `set_trace`:一行代码就地下断点

`ipdb` 是 Python 里最常用的交互式调试器,它的核心就是一句 `ipdb.set_trace()`。你把这一句插到代码里任何位置,程序跑到这儿就会停住,弹出一个交互界面,你可以敲变量名看值、敲 `n` 走到下一行、敲 `c` 让它继续跑。简单讲,`set_trace` 就是手动埋下的『暂停按钮』。比如怀疑回测第 10 天信号不对,就在那行前面写 `import ipdb; ipdb.set_trace()`,跑起来停在那儿,当场把 `signal`、`price` 这些值一个个看个清楚。注意它是交互式的,要在你自己电脑的终端里跑,不能塞进无人值守的脚本里。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

postmortem 事后调试:程序崩了再回去看现场

有时候你没提前设断点,程序自己崩了报错了,难道要从头再来一遍?不用。postmortem,也就是事后调试,可以在程序崩溃之后,直接把你带回出错那一刻的现场,变量的值都还在,就像警察赶到案发现场勘查一样。在 IPython 或 Jupyter 里敲一句 %pdb on,以后只要一报错,它会自动跳进事后调试界面,让你当场查为什么崩。比喻一下:set_trace 是你主动喊停,postmortem 是出了事故之后回去看监控录像,两个都不用从头跑一遍。

CONCEPT_04

logging 分级日志:比满屏 print 优雅得多

print 谁都会用,但项目一大,满屏幕 print 出来的东西分不清主次,想关还得一行行删。logging 就是来解决这个的。它能给每条信息标个等级:INFO 是普通提示、WARNING 是警告、ERROR 是出错了,你可以设一个开关,只看 WARNING 以上的、把啰嗦的 INFO 全部静音,一行代码就能切换,不用删。比喻一下:print 是随手在墙上乱涂,logging 是行车记录仪,自动带时间、带等级、能回放、能一键关掉。比如回测里每天打一条 INFO 记下当天净值,出问题时调成只看 WARNING,屏幕立刻清爽。

CONCEPT_05

未来函数:回测里最常见也最致命的 bug

未来函数,是回测里坑死人最多的一个 bug:你的策略在做今天的决策时,不小心用到了今天还拿不到的、未来才知道的数据。最典型的就是用明天的收盘价来决定今天买不买。比方说你写 `signal = 明天会涨` 这个条件去决定今天买入,那当然每次都买在涨之前,回测里钱越滚越多,净值轻松翻几十倍。可现实里你今天根本不知道明天涨不涨,这种回测全是自欺欺人。识别它的关键就一句话:今天的决策,只能用今天及以前的数据。这一节我们就亲手制造再亲手抓出这个 bug。

04 · PYTHON LAB · 真实可跑代码

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_055_debugging.py — 调试技巧实战:用 logging / assert / traceback 抓出回测里的未来函数 bug
# 真名上屏:ipdb.set_trace() 就地下断点 / postmortem 事后调试(%pdb on)/ logging 分级日志代替 print
# 注意:ipdb 是交互式的,沙盒里不能真跑;下面用 logging + assert + traceback 演示同一套『停下来逐行查』的思路
import logging
import traceback
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs

def _data_path(_name):
    # 铁律62:data/ 放在 notebook 文件夹里。仓库根(run_lab)存取 out/notebook/data/;
    # 原版 notebook 在 out/notebook/ 用 data/;中国版在 out/notebook/cn/ 用 ../data/
    from pathlib import Path as _P
    _here = _P.cwd()
    for _b in [_here/'data', _here/'../'/'data', _here/'out'/'notebook'/'data',
               _here/'../'/'../'/'data', _here/'../'/'../'/'../'/'data']:
        if (_b/_name).exists():
            return str(_b/_name)
    if (_here/'out'/'notebook').exists():
        _t = _here/'out'/'notebook'/'data'
```

```
elif _here.name == 'cn':  
    _t = _here/ '../data'  
else:
```


04 · PYTHON LAB · 真实可跑代码 · 续 (2/6)

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
_t = _here/'data'
_t.mkdir(parents=True, exist_ok=True)
return str(_t/_name)

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False
# 配置 logging:比满屏 print 优雅,每条信息自动带时间和等级,可一键调级别只看 WARNING 以上
logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(levelname)s] %(message)s', datefmt='%H:%M:%S')
log = logging.getLogger('backtest')
CODE, NAME = 'sz.000725', '京东方A'
START, END = '2020-01-01', '2024-12-31'
CSV_PATH = _data_path('D055_ipdb.csv')

# ==== 0. 拉京东方A日线收盘 ====
print('==== 0. 拉京东方A 2020-2024 日线收盘 ====')
import os
if os.path.exists(CSV_PATH):
    # 缓存优先:CSV 在就直接读,不联网
    df = pd.read_csv(CSV_PATH, parse_dates=['date'])
else:
    lg = bs.login()
    if lg.error_code != '0':
        raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
    rs = bs.query_history_k_data_plus(CODE, 'date,close', start_date=START,
                                     end_date=END, frequency='d', adjustflag='2')
    rows = []
    while rs.error_code == '0' and rs.next():
```

```
rows.append(rs.get_row_data())  
bs.logout()  
df = pd.DataFrame(rows, columns=['date', 'close'])
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/6)

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
df.to_csv(CSV_PATH, index=False) # 拉完保存 CSV
df['date'] = pd.to_datetime(df['date'])
df['close'] = pd.to_numeric(df['close'])
df = df.set_index('date').sort_index()
df['ret'] = df['close'].pct_change()
log.info(f'{NAME} 共 {len(df)} 个交易日,收盘从 {df["close"].iloc[0]:.2f} 到 {df["close"].iloc[-1]:.2f}')

# ==== 1. 带 bug 的回测:经典未来函数(信号偷看了明天的涨跌)====
print('\n==== 1. 带 bug 的回测:信号偷看了明天(未来函数)====')
def backtest_buggy(data):
    d = data.copy()
    # bug 就在这一行:shift(-1) 把『明天的收益』挪到今天,等于今天就知道明天涨不涨
    d['tomorrow_ret'] = d['ret'].shift(-1)
    # 用明天会不会涨来决定今天满仓还是空仓 — 现实里今天根本拿不到这个信息!
    d['signal'] = (d['tomorrow_ret'] > 0).astype(int)
    # 策略当日收益 = 今天的仓位 * 明天的收益 — 仓位偷看了明天方向,直接踩明天涨跌交易,几乎天天赚(这就是未来函数的『假涨』)
    d['stgy_ret'] = d['signal'] * d['tomorrow_ret']
    d['nav'] = (1 + d['stgy_ret'].fillna(0)).cumprod()
    log.info(f'带bug 前 5 天信号 = {d["signal"].head().tolist()}(几乎全 1,因为偷看了明天)')
    log.info(f'带bug 最终净值 = {d["nav"].iloc[-1]:.2f} 倍')
    return d

buggy = backtest_buggy(df)
nav_buggy = buggy['nav'].iloc[-1]
ann_buggy = (nav_buggy ** (252 / len(buggy)) - 1) * 100
print(f'带 bug 净值 {nav_buggy:.1f} 倍,年化约 {ann_buggy:.0f}% — 好到不真实,八成是 bug')
```

```
# ==== 2. 用调试手段抓bug:断言 assert + try/except + traceback ====  
print('\n==== 2. 调试手段一:assert 断言兜底,离谱结果当场报警 ====')  
def sanity_check(nav_final, ann):
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/6)

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
# 一条年化不该离谱到几倍的断言:正常 A 股策略年化能有几十个百分点已经很强了
assert ann < 200, f'年化 {ann:.0f}% 离谱到不真实,几乎可以断定有未来函数 bug!'

try:
    sanity_check(nav_buggy, ann_buggy)
    print('断言通过(说明结果还算正常)')
except AssertionError:
    # traceback.format_exc() 把出错现场完整打印出来,就是 postmortem 事后调试要看的『案发现场』
    print('断言失败!下面是 traceback 事后勘查打印的出错现场:')
    print(traceback.format_exc())
    log.warning('assert 抓到离谱年化,启动排查:怀疑信号偷看了未来')

# ==== 3. 调试手段二:logging 对比,证明信号确实偷看了未来 ====
print('\n==== 3. 调试手段二:logging 对比,证明信号用到了明天的数据 ====')
probe = buggy.iloc[5:8][['close', 'ret', 'tomorrow_ret', 'signal']]
for dt, row in probe.iterrows():
    # 关键证据:今天的 signal 跟『明天的收益是否为正』完全一致,说明今天就偷看了明天
    used_future = int(row['tomorrow_ret'] > 0) == int(row['signal'])
    log.warning(f'{dt.date()} 信号={int(row["signal"])} 明天涨跌={row["tomorrow_ret"]*100:.2f}% 是否偷看未来={used_future}')
    print('每一天的信号都精确等于明天的涨跌方向 — 实锤:未来函数')

# ==== 4. 修复版回测:信号只用今天及更早的数据 ====
print('\n==== 4. 修复版:信号只用今天及以前的数据(正确的 shift 方向)====')
def backtest_fixed(data):
    d = data.copy()
    # 修复:用『昨天的收益』判断趋势(今天能拿到),再决定今天仓位 — 绝不碰未来
    d['yest_ret'] = d['ret'].shift(1)
```

```
d['signal'] = (d['yest_ret'] > 0).astype(int)
d['stgy_ret'] = d['signal'] * d['ret']
d['nav'] = (1 + d['stgy_ret'].fillna(0)).cumprod()
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/6)

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
log.info(f'[修复后] 最终净值 = {d["nav"].iloc[-1]:.2f} 倍(回到正常区间)')
return d

fixed = backtest_fixed(df)
nav_fixed = fixed['nav'].iloc[-1]
ann_fixed = (nav_fixed ** (252 / len(fixed)) - 1) * 100

# ==== 5. 对比:带 bug(假) vs 修复后(真)====
print('\n==== 5. 对比:未来函数让回测假涨,一调试就现形 ====')
buy_hold = df['close'].iloc[-1] / df['close'].iloc[0]
summary = pd.DataFrame({
    '版本': ['带 bug(偷看明天)', '修复后(只用今天)', '买入持有'],
    '最终净值(倍)': [round(nav_buggy, 2), round(nav_fixed, 2), round(buy_hold, 2)],
    '年化(%)': [round(ann_buggy, 1), round(ann_fixed, 1), round((buy_hold ** (252 /
len(df)) - 1) * 100, 1)],
})
print(summary.to_string(index=False))
print(f'未来函数让回测净值假涨 {nav_buggy/max(nav_fixed,0.01):.0f} 倍,一上调试工具就现形')

# ==== 6. 画三张图 ====
print('\n==== 6. 画三张图:净值对比 / 调试证据 / 收益对比 ====')
# 图1:带 bug 净值(冲天) vs 修复后净值(正常),叠在一起
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(buggy.index, buggy['nav'], color='#d62728', lw=2, label='带 bug:偷看明天(假涨冲天)')
ax.plot(fixed.index, fixed['nav'], color='#2ca02c', lw=2, label='修复后:只用今天(正常)')
ax.set_yscale('log')
ax.set_title(f'{NAME} 回测净值:未来函数让曲线假涨上天(对数轴)'); ax.set_ylabel('净值(倍,对数)')
ax.legend(); ax.grid(alpha=0.3); plt.tight_layout();
```

```
plt.savefig('nav_compare.png', dpi=120); plt.close()
```

```
# 图2:logging 抓到的调试证据 — 几天的信号 vs 明天涨跌方向,完全吻合=偷看未来  
fig, ax = plt.subplots(figsize=(11, 6))
```


04 · PYTHON LAB · 真实可跑代码 · 续 (6/6)

调试技巧 — ipdb set_trace/ postmortem + logging 分级日志 + assert 断言 + traceback 事后勘查,抓一个未来函数 bug

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
probe_plot = buggy.iloc[5:13]
xpos = np.arange(len(probe_plot))
ax.bar(xpos - 0.2, probe_plot['signal'], width=0.4, color='#1f77b4', label='今天的信号(1=满仓)')
ax.bar(xpos + 0.2, (probe_plot['tomorrow_ret'] > 0).astype(int), width=0.4, color='#ff7f0e', label='明天是否上涨(1=涨)')
ax.set_xticks(xpos); ax.set_xticklabels([d.strftime('%m-%d') for d in probe_plot.index], rotation=45)
ax.set_title(f'{NAME} 调试证据:今天的信号精确等于明天涨跌 = 偷看未来');
ax.set_ylabel('0 / 1')
ax.legend(); ax.grid(alpha=0.3, axis='y'); plt.tight_layout();
plt.savefig('debug_evidence.png', dpi=120); plt.close()
```

图3:带 bug vs 修复 vs 买入持有的最终收益对比柱

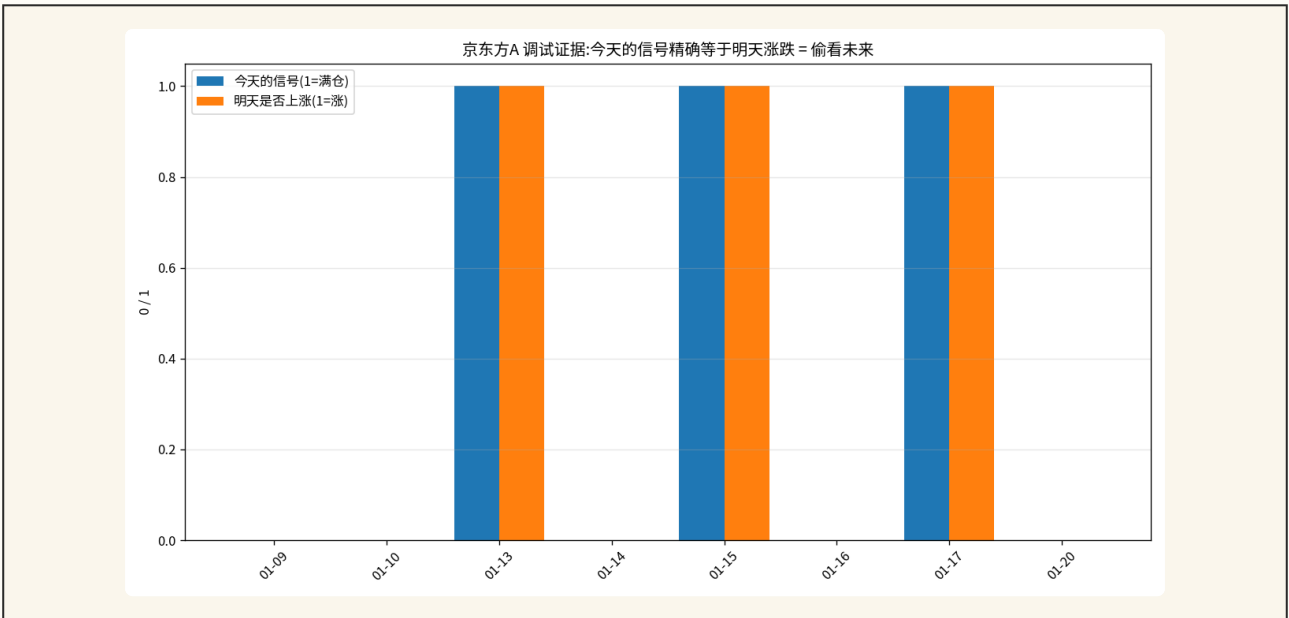
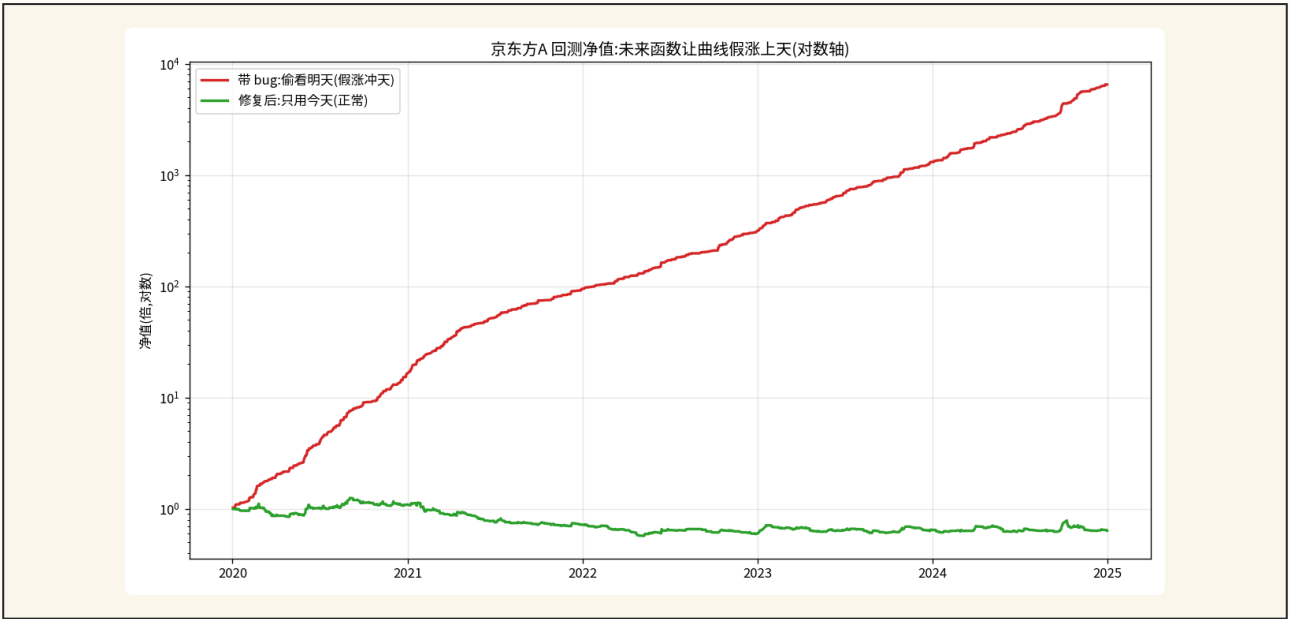
```
fig, ax = plt.subplots(figsize=(11, 6))
labels = ['带 bug\n(偷看明天)', '修复后\n(只用今天)', '买入持有']
navs = [nav_buggy, nav_fixed, buy_hold]
colors = ['#d62728', '#2ca02c', '#9bbbe0']
bars = ax.bar(labels, navs, color=colors)
ax.set_yscale('log')
for b, v in zip(bars, navs):
    ax.text(b.get_x() + b.get_width()/2, v, f'{v:.1f}倍', ha='center', va='bottom')
ax.set_title(f'{NAME} 最终净值对比:未来函数吹出来的假繁荣(对数轴)'); ax.set_ylabel('最终净值(倍,对数)')
ax.grid(alpha=0.3, axis='y'); plt.tight_layout(); plt.savefig('ret_compare.png', dpi=120); plt.close()
```

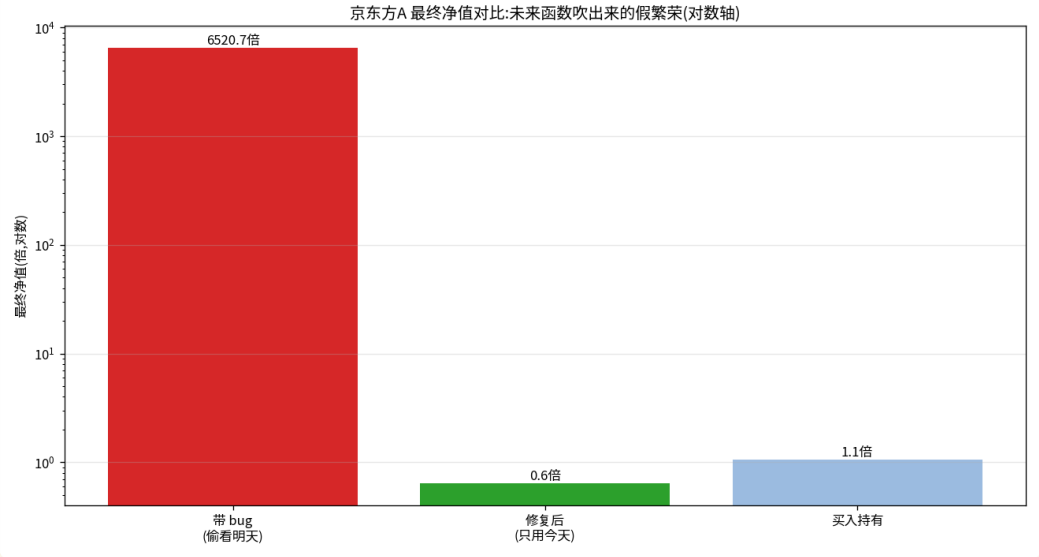
```
print('\n[done] 抓未来函数 bug + 修复 + 三张图完成;真名回顾:ipdb.set_trace 下断点 / postmortem 事后调试 / logging 代替 print')
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





标的与数据(京东方A)

京东方A 2020-2024 五年日线收盘;同一份数据,分别跑带 bug 和修复后两套回测做对照

带 bug 净值(偷看明天)

信号偷看第二天涨跌,净值假涨到 6520 倍、年化约 521%—好到不真实正是最大的红旗

修复后净值(只用今天)

信号改成只用今天及以前的数据,净值回到 0.64 倍,跟买入持有同量级,假繁荣瞬间打回原形

买入持有对照

同一只票什么都不操作一直拿着,五年净值约 1.05 倍;修复后的 0.64 倍跟它是一个量级

assert 报警 + logging 实锤

断言一句『年化超上限就报错』当场抓住 521% 触发 AssertionError;日志打印发现每天信号都精确等于第二天涨跌方向

调试工具(真名上屏)

ipdb 的 set_trace 就地下断点、postmortem 事后勘查;logging 分级日志代替满屏 print;assert + traceback 兜底

► 01 调试不是到处 print,是让程序停在出错那行逐行查

程序出错就到处加 print、满屏打数字,跟车在路上坏了不查、光满大街喊一句哪儿坏了一样,又累又看不出名堂。print 的毛病是你得提前猜对地方、猜错再换、来回十几遍,查完还得一条条删干净。正确姿势是把车架起来在出问题那一刻停下:用断点让程序停在你怀疑的那一处、把那一刻所有变量的值冻住给你看,想看哪个看哪个;没提前设断点、程序自己崩了也不用从头跑,事后勘查能直接把你带回出错那一刻的现场看变量,就像警察赶到案发现场。

► 02 未来函数:回测最常见也最致命的 bug

未来函数,就是你的策略在做今天的决策时,不小心用到了今天还拿不到的、未来才知道的数据,最典型的是用明天的涨跌决定今天买不买。在代码里把一系列数据整体往前挪一格还是往后挪一格,看着就差一个小符号,方向一写反就从只看过去变成偷看未来,而代码本身一点都不报错、照样跑得欢,只是悄悄给你一个假答案。京东方A 这个例子里,信号偷看第2 天涨跌,净值假涨到 6520 倍、年化 521%,把一个其实亏钱的策略包装成了印钞机。识别它只有一句话:今天的决策只能用今天和以前的数据。

► 03 好到不真实就是最大的红旗,assert 替你拉警报

回测跑出年化几百倍这种好到不真实的结果,很多人只顾着高兴,老史就是兴奋得一晚上没睡、第2 天就把大半身家投进实盘,结果连着亏到怀疑人生。对付它最朴素也最管用的一招是断言兜底:给关键结果加一句『年化高到离谱就当场报错』,带 bug 版年化 521% 一跑到这儿立刻触发 AssertionError、逼你停下来查。它最大的好处是不靠你的自觉——人一看赚钱曲线理智就下线,而断言冷冰冰的,数字一越线就拦住你。老史当年要是埋了这么一道断言,就不至于揣着 6520 倍的幻觉冲进实盘。

► 04 logging 实锤:每天信号都精确等于第2 天涨跌方向

光怀疑信号偷看了未来还不够,得拿出铁证。用 logging 把那几天的信号和第2 天到底涨没涨并排打出来对比,你会看到每一天今天的信号都精确等于第2 天的涨跌方向、一天都不差——信号要是没偷看未来,绝不可能天天跟明天的答案对得这么齐。logging 比满屏 print 高明的地方在于:它带时间、带等级、能分类、能对齐,还能一键调到只看 WARNING 让屏幕清爽;换成乱打印,这种诡异的整齐早就淹没在一堆数字里看不出来了。日志不光记,还帮你把真相摆出来。

► 05 修复就一句话,看到好到不真实先警觉而不是先兴奋

修法其实就改一个地方:把信号改成只看今天和以前的涨跌、绝不碰明天,再跑一遍,净值立刻从 6520 倍掉回 0.64 倍,跟买入持有的 1.05 倍是一个量级,假繁荣瞬间打回原形,这才是这只票真实的样子。把这套思路练熟会实实在在改变你的水平:养成看到好到不真实的回测先警觉而不是先兴奋的习惯、给关键结果埋断言、用分级日志代替满屏打印,而最关键那条铁律——今天的决策只能用今天和以前的数据——会像一根弦绷在你每次写信号的时候。这些不起眼的习惯,才是把业余玩家和职业选手真正区分开的东西。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

回测排雷	sz.000725	京东方A 5 年日线,故意写一个偷看明天价格的未来函数回测,净值假涨几十倍,再用调试工具抓出来
日志排查		用 logging 在关键节点打印信号和净值,出问题时调成只看 WARNING,屏幕立刻清爽,一眼锁定可疑那几天
断言兜底		用 assert 给回测加一道防线:年化超过某个离谱阈值就当场报警,逼出八成是 bug 的结果
现场勘查		程序报错时用 traceback 打印完整出错现场,等同 ipdb 的 postmortem 事后调试,不用从头再跑一遍

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

到处加 print, 满屏看不过来

程序一出错就随手加一堆 print, 跑一遍满屏幕都是数, 主次不分、想关还得一行行删, 跟车坏了满大街喊一样低效。正确做法是用 ipdb 在怀疑那一行停下来逐行查, 或者用 logging 分级, 平时只看关键信息, 排查时再放开细节。

△ 02

未来函数自己看不出来

回测里偷看未来是最隐蔽的 bug, signal 用错一个 shift 方向就埋下了, 而且回测结果会特别漂亮, 让人误以为是好策略。判断口诀只有一句: 今天的决策只能用今天及以前的数据。每写一个信号都回头检查一遍它用没用到未来的价格。

△ 03

不设断言, 任由离谱结果蒙混过关

回测跑出年化几十倍这种好到不真实的结果, 很多人只顾着高兴。一定要给关键结果加 assert 断言兜底, 比如年化超过某个阈值就报警, 让离谱结果第一时间现形, 而不是等上了实盘亏了钱才发现。

△ 04

把警告日志当噪音忽略

logging 打出来的 WARNING、ERROR 是程序在向你呼救, 有人嫌烦直接无视。一旦看到 WARNING 就该停下来查, 很多致命 bug 早在日志里报过警, 只是被当成噪音划过去了。日志是用来看的, 不是用来刷屏的。

△ 05

改完不复查, 又引入新 bug

修一个 bug 很容易顺手带出一个新 bug, 比如把 shift 方向改对了, 却忘了同步改后面用到它的地方。改完一定要重新跑一遍、对比修复前后的结果是否合理, 最好再加上断言和日志守着, 别让旧坑刚填好又挖了个新坑。

调试代码的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

01 别到处 print:用 ipdb 在怀疑那行停下来逐行查,用 logging 分级代替满屏打印

02 回测每写一个信号都自查:今天的决策只能用今天及以前的数据,绝不碰未来

03 给关键结果加 assert 断言兜底,离谱数字当场报警,别等上实盘才发现

04 看到 logging 的 WARNING/ERROR 就停下来查,日志是呼救不是噪音

05 修完 bug 必须重跑复查、对比前后结果,别填一个旧坑又挖一个新坑

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 调试的正确姿势不是到处 print,而是让程序在出错那行停下来逐行查现场。
- 02 ipdb 的 set_trace 是手动埋的暂停按钮,postmortem 是程序崩了之后回看现场。
- 03 logging 分级日志比满屏 print 优雅:带时间带等级、可一键调级、想关就关。
- 04 未来函数是回测最致命的 bug:今天的决策偷看了明天的价格,净值假涨几十倍。
- 05 用 assert 断言兜底 + try/except + traceback 现场勘查,离谱结果第一时间现形。
- 06 修复就一句话:信号只用今天及以前的数据,净值立刻从假繁荣回到正常区间。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用『车坏了』打比方,说说为什么到处加 print 不如让程序停在出错那行逐行查。

Q02 ipdb 的 set_trace 和 postmortem 分别是干嘛的?各在什么场景下用?

Q03 什么是未来函数?老史的回测年化几十倍、实盘巨亏,跟它有什么关系?

Q04 assert 断言、logging、traceback 各自在抓 bug 时帮你做什么?为什么修完还要复查?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Python 官方文档 `pdb` (docs.python.org)— 标准库调试器的完整命令手册, `set_trace`、`postmortem`、单步执行全在这里。
- ▶ `ipdb` 项目文档 (github.com/gotcha/ipdb)— `pdb` 的增强版,带语法高亮和补全,讲清 `set_trace` 与事后调试用法。
- ▶ Slatkin 《Effective Python》(2019/Addison-Wesley)— 调试与日志相关条目,讲为什么用 `logging` 代替 `print`、怎么用 `pdb` 排查。
- ▶ Lopez de Prado 《Advances in Financial Machine Learning》(2018/Wiley)— 系统讲回测七宗罪,未来函数与数据泄露是头号陷阱。
- ▶ Python 官方文档 `logging HOWTO` (docs.python.org)— 分级日志入门好文,讲清 `INFO`/`WARNING`/`ERROR` 等级和一键调级的用法。

NEXT LECTURE · 下一节预告

DAY 056 单元测试与策略

学会了出错后怎么抓 bug,下一节我们更进一步:用 `pytest` 写单元测试,在 bug 上线之前就把它拦下来。给你的回测函数写几条小测试,代码一改就自动跑一遍,确保老 bug 不复活、新改动不闯祸,从『出错再救火』升级到『写完就有人替你盯着』。